

tc.)

- Support & troubleshooting of GitLab applications, specifically related to their use of and interaction with the PostgreSQL ecosystem.

- Self-Hosted Clickhouse (coming soon)

Systems or services explicitly not owned by us:

System name	Description	Owner and supported by	Extra info/Open questions
Redis	There are several use cases such as caching, rate-limiting, sidekiq queueing.	Scalability Group	Redis Architecture
Data team systems		Data team	
Self Managed databases		Self managed Support	

Useful Links

Backlog	DBO Status	
Roadmap Ideas	Now-Next-Later	
Reaching us	gdatabaseoperations	@gitlab-org/data-access/dbo
Weekly Agenda	Weekly APAC and EMEA/AMER	

DBO Escalations

We have a detailed DBO escalation process which provides escalation guidelines for handling database related production incidents.

OKRs

We use quarterly Objectives and Key Results to plan and measure our Key Performance Indicators (KPIs).

Performance indicators

We measure the value we contribute by using performance indicator metrics.

In addition to the Infrastructure Department's KPIs for availability and performance of GitLab.com, the Database Operations team tracks the following:

- Backup and Recovery SLOs
- General database availability(Uptime)

Key Technical Skills

The team is comprised of Database Operations engineers with varying levels of expertise in:

- Supporting PostgreSQL in large production environments.
- Infrastructure automation and configuration management, using tools such as Chef, Ansible,

Terraform, etc.

- PostgreSQL internals, tuning & optimization, SQL and PL/pgSQL.

Common Links

To make it easier to find your way around you can find a list of useful or important links below.

Monitoring & Performance Related Tools

The following tools can be helpful:

- Postgres Checkup: Detailed report about the status of the PostgreSQL database.
- Private Grafana: for both application and system level performance data.
- Performance Bar: type pb in GitLab and a bar with performance metrics will show up at the top of the page. This tool is especially useful for viewing the queries executed and their timings.

If you have identified a Postgres Performance Problem and want to dive deeper, we recommend these runbooks and dashboards. They are best used together, with the runbooks explaining how to understand the dashboards.

Dashboards:

- Postgres node performance overview (high-level)
- Postgres aggregated query performance analysis
- Postgres single query performance analysis
- Postgres wait events analysis

Runbooks:

- High-level performance analysis and troubleshooting of a Postgres node
- Postgres wait events analysis (a.k.a. Active Session History; ASH dashboard)
- SQL query analysis and optimization for Postgres

Dashboards

The following (private) Grafana dashboards are important / useful for database specialists:

- PostgreSQL Overview
- Patroni Overview
- Patroni CI Overview
- PgBouncer Overview
- PgBouncer CI Overview
- GitLab Triage
- PostgreSQL Bloat
- PostgreSQL Disk IO
- Host stats
- Tuple Statistics
- Postgres node performance overview (high-level)
- Postgres aggregated query performance analysis

- Postgres single query performance analysis
- Postgres wait events analysis

Documentation

- What requires downtime?
- Adding database indexes
- Post Deployment Migrations
- Background Migrations
- SQL Migration Style Guide
- SQL Query Guidelines
- Infrastructure runbooks and documentation

SECTION: engineering/infrastructure-platforms/data-access/database-operations/dbre-escalation-process.md

```
{{% alert title="Note" color="danger" %}}  
We are using PagerDuty for escalations.  
{{% /alert %}}
```

About This Page

This page outlines the DBO team's incident escalation policy.

Shortcuts

DBO PagerDuty schedule

Slack x PD integration: /pd trigger @dbo-oncall (For urgent reach-outs, ALWAYS use PagerDuty)

Slack handles: @dbre or @dbo-oncall (Non-urgent)

Slack channels: gdatabaseoperations

group::database operations

Production Incidents

SLO and Expectations

DBO RESPONSE IS ON A BEST-EFFORT BASIS

LOCAL TIMEZONE, WEEKDAY COVERAGE ONLY

S1 / S2 INCIDENTS ONLY

NB1: Due to limited staffing, i.e. having only one person in EMEA timezone, there will be times during the business day, within multiple timezones, where there will not be anyone able to respond. We understand the criticality of responding to S1/S2 incidents and we will make every effort to ensure there is adequate and timeliness in our responses, but given the current staffing levels, we are not at this point adhering to a hard SLO. To do justice to this situation, it is also expected that schedules are changed on an ad-hoc bases.

NB2: DBO will join incidents as a subject matter expert in a consultative capacity and there should be no expectation that the DBO engineer is solely responsible for a resolution of the escalation. There may be times where the DBO needs to escalate to other subject matter experts, such as the Database Framework (DBF) team, in order to make headway on the incident at hand.

Escalation Process

Scope and Qualifiers

1. GitLab.com S1 and S2 production incidents raised by the Incident Manager On Call, Engineer On Call and Security teams.

NB1: Gitlab Dedicated support is consultative at this point. DBO team currently not equipped, i.e. lacking access and training on how to support Dedicated databases. This may change in the future; check back here for updates on this topic.

NB2: Self Managed support is discretionary and will be evaluated on a case-by-case basis.

NB3: This process is NOT a path to reach the DBO team for non-urgent issues. For non-urgent issues, please create a Request for Help (RFP) issue using this Issue template.

NB4: The DBO on-shift is responsible for coordinating warm handoffs during shift changes, especially when there is an ongoing, active incident.

Escalation

1. EOC/IM, Development or Security page the DBO on-call via PagerDuty
1. DBO responds by acknowledging the page and joining the incident channel and zoom
1. DBO triages the issue and works towards a solution.
1. If necessary, DBO reach out for further help or domain expert as needed.

NB1: If DBO support does not respond, escalation path as defined within PagerDuty ensues.

Resources

Responding Guidelines

When responding to an Incident, utilize the below procedure as guidelines to follow to assist both yourself and the members requesting your assistance

1. Join the Incident Zoom - this can be found bookmarked in the relevant incident Slack Channel
1. Join the appropriate incident slack channel for all communications that are text based - Normally this is inc-<INCIDENT NUMBER>
1. Work with the EOC to determine if a known code path is problematic

Should the knowledge of this be in your domain, continue working with the EOC to troubleshoot the problem

Should this be something you may be unfamiliar with, attempt to determine code ownership by team - Knowing this will enable us to see if we can bring online an Engineer from that team into the Incident

1. Work with the Incident Manager to ensure that the Incident issue is assigned to the appropriate Engineering Manager - if applicable

Shadowing An Incident Triage Session

Feel free to participate in any incident triaging call if you would like to have a few rehearsals of how it usually works. Simply watch out for active incidents in incidents-dotcom and join the Situation Room Zoom call (link can be found in the channel) for synchronous troubleshooting. There is a nice blog post about the shadowing experience.

Replaying Previous Incidents

Situation Room recordings from previous incidents are available in this Google Drive folder (internal).

Shadowing A Whole Shift

To get an idea of what's expected of an on-call DBO and how often incidents occur it can be helpful to shadow another shift. To do this simply identify and contact the DBO on-call to let them know you'll be shadowing. During the shift keep an eye on incidents-dotcom for incidents.

Tips & Tricks of Troubleshooting

1. How to Investigate a 500 error using Sentry and Kibana.
1. Walkthrough of GitLab.com's SLO Framework.
1. Scalability documentation.
1. Use Grafana and Kibana to look at PostgreSQL data to find the root cause.
Related incident: Postgres transactions timing out; sidekiq queues below apdex score; and overdue pull mirror jobs.
1. Use Grafana and Prometheus to troubleshoot API slowdown.
Related incident: 2019-11-27 Increased latency on API fleet.
1. Let's make 500s more fun

Tools for Engineers

1. Training videos of available tools
 1. Visualization Tools Playlist.
 1. Monitoring Tools Playlist.
 1. How to create Kibana visualizations for checking performance.
1. Dashboards examples, more are available with the dropdown list at upper-left corner of any dashboard below
 1. Saturation Component Alert.
 1. Service Platform Metrics.
 1. SLAs.
 1. Web Overview.

SECTION: engineering/infrastructure-platforms/data-access/durability/_index.md

Mission

The mission of the Durability team is dedicated to safeguarding and securing customer data that is stored by the GitLab application and set guidelines for data access.

We strive to build and maintain resilient infrastructure and improve the management of Redis, Sidekiq, and Gitaly.

Ownership

The team has ownership over the following areas of the GitLab product:

1. Reliable backup and restore solutions for all environments where GitLab is deployed.
2. Data management and performance for Sidekiq and Redis.
3. Infrastructure support for the Gitaly service.

Services

Durability is responsible for infrastructure that supports the following GitLab application services:

1. Sidekiq
2. Redis
3. Gitaly

Getting Assistance

Should you require assistance from the Durability team, please reference getting assistance in Infrastructure

We also have team handles that ping the full team:

- Slack: @infra-durability
- GitLab: @gitlab-org/data-access/durability

Priority Levels for Requests

The Durability uses a single priority label ~"Durability::P1" for very urgent work that is blocking significant other work.

For issues with this label we aim to engage within 24 hours.

Team Members

{{< team-by-manager-slug "jarv" >}}

Common Links

- Durability Top Level Epic
- gdurability - work related discussions, external requests, etc

How We Work

Project Management

The Durability team has a single top level Epic that links sub-epics for all projects the team is working on.

Every sub-epic has a completion date and a DRI for providing a weekly status update. We follow Platforms Project Management practices as outlined in the Handbook.

Issues

Issues for work performed by the Durability team is located in the Durability team issue tracker.

The exception to this is work that is part of the Rails, these issues should be opened in the GitLab Rails issue tracker.

SECTION: engineering/infrastructure-platforms/data-access/git/_index.md

Mission statement

The Git team is responsible for building, maintaining and providing expertise on the Git version control system. Its main responsibilities include:

- Upstream development of the Git version control system.
- Provide expertise to other teams at GitLab.
- Foster the Git community.
- Ensure the long-term viability of the Git project.

Upstream development

The Git team is responsible for driving the upstream development of Git both in accordance with the goals of the community and to address GitLab-specific needs as raised by other teams. This falls into the following broad categories:

- Implementation of new features in the form of new tooling as required by use cases at GitLab.
- Scalability improvements in the form of optimizations and new data formats.
- Improvements to the usability of Git so that it remains the first choice for developers.
- Maintenance of the Git codebase.

Provide expertise

The Git version control system is at the core of what GitLab is doing and is thus of critical importance to many teams across GitLab. This doesn't only include teams part of the Engineering division, but also spans across other divisions like for example Support. The Git team is the main contact point for such teams and provides expertise on how to use Git effectively and efficiently. It will provide insights into following non-exhaustive list of topics:

- How can a certain use case be implemented with Git?
- How to improve performance of certain workloads?
- How to expose certain features of Git to customers?
- What is happening in the Git community and what is the direction the project heads into?
- Which Git related workflows and practices should be encouraged among Git users?

Foster the Git community

GitLab relies on the Git community to be healthy. It is our belief that a healthy community is a community with diverse view points, including those views held by our direct competitors active in the same community. The Git team will help foster and grow the Git community:

- Ensure an influx of new contributors via mentorship programs like Google Summer of Code or Outreachy.
- Improve Git documentation and materials for new contributors.
- Attend or host community-oriented events like user groups or Git Merge.
- Help out community members by providing guidance and doing reviews.
- Spread information in the form of for example blog posts or Git Rev News.

Long-term viability

The Git project has been growing organically for almost two decades at the point of writing. It has thus accumulated a lot of debt in different forms that need to be addressed in one form or another to ensure that the project itself remains viable in the long term. This includes topics like:

- Maintenance and refactorings of the Git codebase to reduce technical debt.
- Modernization of parts of the tech stack.
- Improvements to the usability of Git.
- Deprecations and removal of features.

Roadmap

Please see the public product direction for GitLab.

The current roadmap is this epic board.

Team

Responsibilities |
In the product hierarchy

```
{{< team-by-departments "Git Team" >}}
```

How to contact the team

We are happy to help when other teams or customers need it. But please keep in

mind that we are primarily a development team, not equipped for "field engineering".

Our engineers can help, preferably asynchronously, with deep technical investigation based on Git-specific use cases and data and able technical collaboration, in close partnership with Support, CSM and direct users of Git like Gitaly.

Engineering Managers (@pks-gitlab) and Product Managers (@mjwood) are also happy to engage with customers if you need assistance clarifying roadmaps, features and timelines, or to ensure the correct prioritization.

We are not a good fit however if you need:

- Guidance on how Git is used by other teams. While we can provide the expertise on how to integrate it, the actual integration is outside of our reach.
- Advice on workflows and instance configuration or architecture in self-hosted scenarios. (Reference Architectures and Professional Services can help)
- Engagements without clear exit criteria. Please clarify them first, "let's jump on a call to discuss" is usually in this category.
- Long-term "advise us" scenarios. Please refer to Support and the documentation, or engage Professional Services.

Git Office Hours

The team regularly hosts GitLab-internal Git Office Hours.

The event is part of the Gitaly team calendar, is announced in due time and typically happens on Wednesday at 2:30pm UTC.

These office hours are mostly used to discuss upstream development in the Git team, but we also encourage other people not part of that team to join in case they want to learn about what is going on in the Git community.

This venue can be used to ask for guidance on specific issues:

- Ask for feedback and guidance on how to integrate Git into the product.
- Discuss interesting edge cases as hit e.g. by our customers.
- Provide feedback on Git features.

Note that this is not intended as a general support forum for how to use Git on a day-to-day basis. A better channel to ask such questions would be [git-help](#).

Please feel free to chime in and add your topic to the agenda linked above.

Bugs, features, performance

To get Git team work on something, it's best to create an issue on the GitLab-specific Git issue tracker and add the `group::git` and `workflow::problem validation` labels, along with any other appropriate labels. Then, feel free to tag the relevant Product Manager and/or Engineering Manager as listed above.

For information requests and other quick one-offs, feel free to use `ggit` on Slack to get attention on the issue.

Urgent issues and outages

If you're not part of the Support organization, please consider seeking help from them first -- Support has better availability and can help in most common cases.

If you still need help, please file an issue here. Post it on `ggit` for more immediate visibility and tag EM and PM, and the Support person you're working with.

On-call rotation

Members of the Git team participate in Gitaly on-call rotation. For more details, see the team page.

Workflow in upstream Git

Because the Git project is an upstream project, we must use a different workflow.

Planning

The Git project has a different release schedule where a new release is published roughly every three months. The schedule can be seen in the Git project's calendar. Furthermore, it is expected that most topics will take significantly longer both to develop and to be merged upstream. As a consequence, our usual milestones are not a good mechanism to properly track the target GitLab version in the context of upstream issues.

Instead, we use labels like `git-milestone::v2.42` to annotate our target Git version for open issues as well as the actual version something has landed in for completed issues. These milestone labels are applied in addition to our normal milestones.

This means that we assign milestones as a timeframe to schedule the implementation of the work by the Gitaly team members, but this does not correspond to the timeframe when the work will be released by the Git project. Their meaning is reduced to a planning tool such that Git issues continue to show up on usual issue boards and are readily discoverable. Consequentially, our usual way of picking up new work applies to Git issues, as

well.

Differences to normal review process

When working with the upstream Git Project, we still use a simple workflow where the developer implements the issue in a topic branch, creates a merge request, and assigns reviewers as usual, where issues and merge requests should be created in our mirror of the Git project. However, because the changes must undergo upstream review, the workflow diverges from our normal workflow

- Changes may optionally undergo internal review.
- After the optional internal review, patches are sent to the Git mailing list.

The meaning of the merge requests thus gets reduced to be purely informational.

Developers are encouraged to skip the optional internal review and instead send their patches to the mailing list directly. In that case, they should add a comment to the merge request that links to the mailing list thread on <https://public-inbox.org/git/>. The review should then happen on the mailing list directly.

Encouraging developers to send patches to the mailing list directly has multiple benefits:

- Reviewers get exposure to the Git mailing list. This demonstrates to the wider Git community there is a whole team working on Git at GitLab while also building up credibility for the reviewer.
- Topics which may otherwise not get a lot of attention may get more when our own reviews are visible.
- We avoid having to do two sequential reviews, which would otherwise further delay topics.

The optional internal review is specifically with new internal contributors in mind. It should help to lower the bar for entry and reduce the level of discomfort that wider exposure to the Git community may bring with it. In the same spirit, reviewers may post review comments internally first before sending them to the Git mailing list. Ultimately, the goal should be that we mentor these new contributors internally such that they eventually start to post to the Git mailing list directly.

The following is a non-exhaustive list of exceptions where we prefer internal reviews:

- Security issues are implemented in our private security mirror.
- Potentially controversial topics where we need to assess whether a proposed solution addresses our own usecases.

Long-running topics

In some contexts it may be necessary to collaborate on long-running topics when it is

infeasible to land changes in a single release. Our usual iterative development approach does not always work well within the Git project. Most importantly, for an open source project the assumption needs to be that contributors may go away in the middle of their endeavour, leaving behind dangling pieces that might have complicated some of our architecture without actually reaping the intended benefits. In contrast to company-led projects, the Git project has no way of ensuring that these would eventually get finished.

For topics that span over multiple releases we are forced to use long-running feature branches until the topic becomes upstreamable. The workflow here is thus different to normal upstream work and happens on a separate topic branch that the developer creates from the current upstream master branch. From here on, the development of the topic happens internally and follows our usual merge request workflow including internal reviews, where each of the merge requests targets the topic branch.

One problem in this context is that the topic branch will eventually start to become outdated when the upstream master branch progresses. While it would be possible to regularly merge from master into the topic branch, the end result would not be in an upstreamable state. On the other hand, rebasing the topic branch on top of master would require other contributors to adapt to the resulting forced update.

Instead, we use a combination of rebases and merges to update the topic branch with upstream's master branch:

1. The topic branch is fake-merged into the master branch, discarding all changes. The resulting tree is the same tree as the tree of master.
1. All commits of the topic branch are rebased on top of the fake merge.
1. Optionally, the rebased commits are squashed or reorganized so that the result becomes upstreamable.

While complicated, this workflow provides a bunch of benefits:

- We never need to force-update the topic branch.
- The topic branch remains in an upstreamable state.
- The history of the topic branch remains intact, which also means that individual contributions remain intact even if they are squashed into other commits.

This workflow is similar to what Git for Windows uses and can be implemented via its shears.sh script:

shell

Update origin, pulling in new changes for origin/master

```
$ git fetch origin
```

Switch to the topic branch

```
$ git switch topic
```

Find the current root of the topic branch. This could either be the last fake-merge

```
$ base=$(git rev-parse ':/Start the merging-rebase')
```

Or it could be the merge-base of origin/master and your topic branch

```
$ base=$(git merge-base origin/master topic)
```

Rebase the topic branch onto origin/master

```
$ ./shears.sh --merging --onto origin/master $base
```

Iteration

The Git mailing list is commonly perceived as daunting due to the fair amount of bikeshedding and criticism you will often receive as response to a patch series. This is not a bad thing though: if used correctly, this feedback mechanism can be used as a tool to help yourself evaluate ideas and avoid design issues. For this mechanism to work though it is important to iterate on patch series:

- Avoid perfectionism, especially with larger patch series before sending them to the mailing list. The chance is high that you will receive feedback that requires you to reconsider basic design ideas, and that would cause a lot of the time you have spent with polishing to be lost.
- You do not need to have all answers before sending a patch series to the mailing list. Instead, it is fine to explicitly highlight areas that you are unsure about and ask for feedback there.

For these mechanisms to work to your benefit you need to engage with the extended community:

- You should send a new version of a patch series at most a couple of days after having received feedback. This ensures that discussions on the previous version thereof are still fresh in the minds. Furthermore, patch series that show a lot of activity are likely to draw more attention from other contributors, as well.
- Respond to feedback that you have received as fast as possible. A fast exchange is a prerequisite for a fruitful discussion and ensures that you keep momentum.
- On the other hand, it shouldn't be necessary to respond to every small typo correction in case you will send out the next version soon anyway. If it will take a while before you send the next version though it is nice to acknowledge nits, but mention that you will hold off sending a new version of the series until you got more feedback.
- Consider the viewpoint of the other person and be ready to disagree and commit. Do not ignore feedback that you have received, as that will lead to frustration and a decreased likelihood for that person to review your future patch series.

Build trust by being a good citizen

As with most open source projects, the foundations of the Git community are built on trust. This is only natural: while companies can exert control over the direction of a project as well as on who is working on them, open source projects have to rely on intrinsic motivation of individual contributors.

This motivation may easily change though based on individuals having less free time or because companies refocus their efforts. The consequence may be that large projects are never finished. As such, being trusted and dependable is quite important in the Git community as it allows you to tackle larger projects.

Unfortunately, trust is a resource that takes significant time to build. It is a limiting factor to what developers can achieve. This is a key difference with other GitLab-controlled projects: you cannot just add an engineer to the Git team and expect them to land large projects. These people will first have to spend some time with the Git community to demonstrate their expertise.

Next to advancing the state of Git itself, there are a couple of other important measures to help build trust:

- Review patch series from other individual contributors not part of GitLab's Git team, even when those are outside of GitLab's current focus area. This helps you build better relationships with them, and as reviewers are scarce it also helps the Git community.
- Critically review patch series from fellow team members on the Git mailing list. We should demonstrate that we hold each other accountable, as well, instead of just rubber-stamping our patch series.
- Engage in general discussions and bug reports.
- Take responsibility of the codebase. If you see something that could be improved, it may be a good idea to address it. On the one hand this ensures that the Git codebase improves over time. On the other hand it is another good mechanism to get the goodwill of the community.
- Keep your promises. If you say that you will follow up on a specific item, e.g. as part of a review, then you should keep that promise. This shows other community members that you are dependable and allows you to tackle larger problems that may require multiple steps.
- Be consistent with your presence on the Git mailing list. If you are being copied on mails it is very likely that you should have a look and reply. This gives others the feeling that you are dependable as a subject matter expert.

Transparency

One large benefit that GitLab has over other code forges that participate in Git development is that most of our planning and coding happens in the open. This allows us to be much more open about why we want to implement certain features in Git, as well.

In general, it is thus recommended to be very explicit about the specific issue that we face and how a patch series would address that issue. We can directly

link to supporting code and issues in Gitaly to give more context. We may also share information about specific metrics, unless there is any kind of customer data involved.

SECTION: engineering/infrastructure-platforms/data-access/gitaly/_index.md

What is Gitaly?

The Gitaly team is responsible for building and maintaining systems to ensure that the Git data storage tier of GitLab instances, and GitLab.com in particular, is reliable, secure and fast. For more information about Gitaly, see the README in the repository and the roadmap below.

The team includes Backend Engineers and SREs collaborating to deliver a reliable, scalable and fast data storage to our customers.

Functional boundary

While GitLab is the primary consumer of the Gitaly project, Gitaly is a standalone product which can be used external to GitLab. As such, we strive to achieve a functional boundary around Gitaly. The goal of this is to ensure that the Gitaly project creates an interface to manage Git data, but does not make business decisions around how to manage the data.

For example, Gitaly can provide a robust and efficient set of APIs to move Git repositories between storage solutions, but it would be up to the calling application to decide when such moves should occur.

Processes fully independent of business inputs (such as repository maintenance) should be fully contained within Gitaly as they provide substantial value to anyone using the Gitaly project.

Roadmap

Please see the public product direction for Gitaly.

The vision and principles driving the roadmap can be found in the internal handbook.

The current roadmap is this epic board. See Roadmap planning below as to how this is managed.

Featured upcoming large architectural changes

- Iterate on the design of object pools
- Rework repository backups
- Gitaly Adaptive Concurrency Limit
- Handle upload-pack traffic in a pure HTTP/2 server
- Transaction management in Gitaly

Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts role="[&.] Systems:Gitaly( API)?" manager-role="Backend Engineering Manager, Gitaly" >}}
```

How to contact the team

Urgent issues and outages

If you're not part of the Support organization, please consider seeking help from them first -- Support has better availability and can help in most common cases.

If you still need help, please file an issue here. Post it on ggitaly for more immediate visibility and tag EM and PM, and the Support person you're working with, and @tier2-oncall-gitaly to notify the Gitaly team member who is on call.

On Call Rotation

Gitaly on-call should only be paged by the following people:

- SRE on-call or IMOC during production incidents only.
- Support Engineers or Support Managers during customer emergencies.

Use /incident escalate on Slack for these cases, then select the Gitaly EOC under On-call teams.

For all other cases please file an issue under Customer issues.

Please do not page on-call outside of these cases. If you're working on a customer emergency but not part of Support, please contact Support instead!

Rotation

The incident.io schedule is the source of truth for who is on-call.

The rotation is staffed during working hours of team members (no weekends). This still covers 24h of workdays, given the distribution of team members, but without guarantees.

- Weekends are explicitly out of scope (not staffed), and escalation must fall back to the current EOC rotation.
- Given that responsibilities are only during working hours, there's no additional compensation unless explicitly specified otherwise.
- You can choose to take time in lieu via Workday, selecting the On-Call Time in Lieu option after a shift.

Expectations during on-call shift

- Refer to the Responder Quick Start Guide for a streamlined onboarding process.

Note : All escalations to the Gitaly team will be made via incident.io

- 15 minutes response time to a incident.io page while on-call. This does not apply to pings to the @tier2-oncall-gitaly Slack handle, which should be used to inform the Gitaly on-call of relevant happenings, but should not be used for emergencies.
- The on-call is expected to be available and reachable (but not necessarily actively working, as long as you can start the investigation within this SLO.)
- If paged less than 15 minutes before the end of a shift, you still must respond and explicitly hand off the incident.
- Serve as point of contact for questions in the ggitaly channel as well as new Request For Help issues.
- Acknowledge inquiries in the ggitaly channel on a best-effort basis.
- Triage new Request for Help issues: establish urgency and work with EM/PM to assign a milestone.
- Ongoing production incidents and customer escalations are explicitly handed off by the outgoing on-call to the next Gitaly on-call using the incident channel on Slack.
- Team members are responsible for finding coverage for PTO and Holidays. Install incident.io mobile application, navigate to Schedules then click on the person icon with arrows to request for cover

Customer issues

Please file an issue here. Post it on ggitaly for more immediate visibility.

A note on customer escalations and engagements

We are happy to help when a customer needs it! But please keep in mind that we are primarily a development team, not equipped for "field engineering".

Our engineers can help, preferably async, with:

- deep technical investigation based on data and able technical collaboration, in close partnership with Support and CSM
- providing product-level fixes or improvements, work to be scheduled and results released as usual, under direction of EM and PM
- improving our documentation if something's unclear

Engineering Managers (@jcaigitlab) and Product Managers (@mjwood) are also happy to engage with customers if you need assistance clarifying roadmaps, product features and timelines, or to ensure the correct prioritization.

We are not a good fit however if you need:

- advice on GitLab instance configuration or architecture in self-hosted scenarios (Reference Architectures and Professional Services can help)
- engagements without clear exit criteria (please clarify them first, "let's jump on a call to discuss" is usually in this category)
- long-term "advise us" scenarios (please refer to Support and the documentation, or engage Professional Services)

This epic discusses possible development of this engagement model.

Normal priority requests

To get Gitaly team work on something, it's best to create an issue on the Gitaly issue tracker and add the group::gitaly and workflow::problem validation labels, along with any other appropriate labels. Then, feel free to tag the relevant Product Manager and/or Engineering Manager as listed above.

For information requests and other quick one-offs, feel free to use ggitaly on Slack to get attention on the issue.

Issues with Infradev labels

These are typically Corrective Actions or other followup items that have strict SLO tracking. They will be scheduled through either of the above paths, by EM and/or PM polling these dashboards.

Training material

- <https://handbook.gitlab.com/handbook/engineering/on-call/expectations-for-on-call>
- Debugging Gitaly
- Production Training issues

Roster management

Please refer to <https://handbook.gitlab.com/handbook/engineering/on-call/pagerduty> for the mechanics (swapping on-call, adding new team members to the rotation).

Team Members

{{< team-by-departments "Gitaly Team" >}}

Working with product

Agile workflow in Gitaly

We generally follow the Product Development Flow to schedule and track our work.

Work is executed in small chunks (2-3 days of work), each tracked as an issue. This allows for natural "checkpoints" for safe context switching.

Triaging and scheduling is separate from executing the current work. All incoming work is tracked and we are intentional about picking up new work.

Incoming work of all kind (both projects and ad-hoc interrupts) passes by EM and PM for triage. There may be some engineering consultation here about feasibility, fit with the product's strategy roadmap etc. Some will get scheduled, some goes to the backlog. If the effort is not deemed necessary or not believed to align with the roadmap, we will close the issue with commentary as to why it is not being pursued for future reference.

We aim to scope milestones such that we have a task list that is ambitious, but not overwhelming. We deliberately leave some capacity for incoming incidents. We want to avoid the feeling of a never ending mountain of work to promote a healthy work / life balance. It is also important to stress that milestones are recommendations only and we work on a best effort basis.

For issues with a strict SLO, we follow the process defined below

We use the following workflow labels on the issues:

- 1. workflow::problem validation - A good spot to put features that we may / may not want to pursue. This is where product can do some user interviews, cost analysis, market fit, etc to decide if it's an opportunity we wish to pursue.
- 1. workflow::solution validation - Use this label for features / issues where Engineering needs to investigate / propose a solution going forward, or break it down into smaller issues.
- 1. workflow::planning breakdown - Issues ready to be scheduled in the next few milestones (unblocked or soon unblocked, with a known solution). Leaders of long-running (pre-approved) projects use this to communicate with PM.
- 1. workflow::ready for development - Work that is scheduled for a milestone (either the current one, or one in the future).
- 1. workflow::in dev - Actively being worked by the Engineering team
- 1. workflow::in review - Work that is in review
- 1. workflow::verification - code is in production and pending verification by the DRI engineer
- 1. workflow::complete - changes are verified, issue can be closed

Issues that we definitely want to prioritize for a release receive a Deliverable label and are moved to the top of the list.

These Deliverable issues help show our commitment to GitLab and our customers around working on these issues.

Workflow

Project Work

The top level Gitaly epic contains linked epics representing projects the team is working on. Team members will either be the primary owner of an epic, or a supporting contributor. This way knowledge gets shared across the team.

DRI & Supporting contributors

The DRI of an epic will be responsible for making decisions regarding technical direction of a project. Making a decision will involve creating proposals and gathering feedback from peers and the Engineering Manager. It also involves reaching out and collaborating with stakeholders external to the team when applicable.

The DRI is also responsible for project management, which means keeping the epic up to date with relevant issues, removing issues that are no longer relevant, and writing weekly updates the automatically generated comment in the epic with the following format:

markdown

HIGHLEVELSUMMARY

:tada: achievements:

-

:issue-blocked: blockers:

-

:arrowforward: next:

-

The supporting contributor(s) of an epic will be responsible for supporting the DRI in working on issues, reviewing MRs, and participating in technical discussions. The secondary owner can also act as the primary owner when the DRI is OOO, depending on their bandwidth.

Supporting contributors are highly recommended but optional. There can also be multiple secondary owners for a project.

Not everyone needs to be a DRI, but everyone should be a supporting contributor on at least one project.

The structure of having both DRIs and supporting contributors does not introduce any hard requirements for moving MRs forward, as reviews and approvals can be done by anyone on the team.

Technical Roadmap, Customer Issues, and Cross Functional Issues

The Gitaly Technical Roadmap & Customer Issues board contain one-off issues that are not a part of any projects, but are important issues to address. These include technical roadmap issues, customer issues, and cross functional work in Gitaly that other teams rely on. These issues will be sorted by priority. Team members can pick up work from this board in addition to issues they are working on as part of project epics.

As a rule of thumb, the ratio of project work to technical roadmap, customer issues should be roughly 70/30.

Urgent and high priority issues

P1/S1 issues should be treated with urgency. If such issues have not been scheduled, bias for action is encouraged.

Go ahead and pull them into the current milestone, but do notify the EM and PM.

Blocked issues

If your work is blocked, use `workflow::blocked` and set a blocking issue for clarity. Then consider asking for help and/or helping to unblock another team member's blocked work before picking up something else.

Issues blocked for a long time should be removed from this process by removing the milestone and unassigning.

Adding more work for the team

Everyone can file new issues as more work is discovered, and feed them into this process. To do so, file an issue, tag EM and PM, and assign `workflow::planning breakdown` without a milestone. Please explain both what needs to be done and why (ie the impact and urgency), and make it clear whether the work is ready to be picked up. (This is also how project DRIs add the next steps in their projects to the workflow.)

Meta

A weekly call is held between the product manager and engineering managers (of both Gitaly and Git teams). Everyone is welcome to join and these calls are used to discuss any roadblocks, concerns, status updates, deliverables, or other thoughts that impact the group.

Roadmap planning

The current roadmap is this epic board. It consists of themes/projects running for a quarter or longer (in some cases, much longer). It is okay to add sub-projects directly to the roadmap in the latter case.

- Anyone can propose a project: file an epic and discuss with the team (and EM+PM). Don't forget the group::gitaly label.
- Once accepted, we add the Roadmap label.
- Ongoing roadmap items get `roadmap::now`, while `roadmap::next` and `roadmap::later` show what's been triaged and pushed into the future for now.
- At each quarterly planning:
 - we review roadmap items (using arguments from the vision and principles, current business priorities etc)
 - and then take on OKRs that push those goals forward.

Quarterly Planning

Quarterly planning is done before every quarter for the next 3 milestones, with input from everyone. At that time, we must already have a good idea of the work that needs to be done.

The process is as follows:

1. EM+PM (with input from engineers and stakeholders): decide the scope we'll be working on, which will align with department level OKRs.
1. EM+PM+Engineers: Based on roadmap items, file smaller epics/issues if needed that can be completed in 3 milestones (ie one quarter). Tie them to the overall project epics. This is where we'll track the actual work.
1. EM: Modify the top level Gitaly epic to reflect the work.
1. PM: Once the scope of the quarter is clear, take the list of issues and assign one of the three milestones, along with `workflow::planning breakdown (for large issues in need of breakdown) or workflow::ready for development`.
1. Engineers: help break down workflow::planning breakdown items and file smaller issues if needed, adding them to the same 3 milestones as reasonable. Raise exceptions as needed.

Handling issues with strict SLO

Issues with Infradev label are typically Corrective Actions or other followup items that have strict

SLO tracking. They will be scheduled through either of the above paths, by EM and/or PM polling these dashboards:

Infradev Dashboard

Past due Infradev issues

1. EM+PM: Poll the dashboards at least weekly. Triage and schedule these issues so that SLOs can be met. If needed, move the issue to the Gitaly tracker, or file a proxy issue there so that it shows up on work boards, and mark it as blocking. Drag issues to the top of the workflow::ready for development column.
1. EM+PM: If the issue is blocked or depends on ongoing work, add a Milestone that fits the SLO and the pending work (so that we don't forget it). Ensure that blocking work gets scheduled before.
1. Engineers: please prioritize picking up this work, and post frequent (at most weekly, even if no changes) updates in the original issue. Mark any blocking issues as such.

Gitaly consumers

To have a constant communication flow about planned changes, updates and maybe breaking changes we have the ggitaly Slack channel. In the channel we will provide updates for all teams using the service but also ask for assistance to provide feedback and insights about planned changes or improvements.

To support this pro-active communication additionally there is also an individual counterpart on the consumer side to help with research in the codebases and

coordination with all the teams consuming Gitaly. The DRI on Consumer side is Igor Drozdov.

The Gitaly consumers are:

- GitLab Rails
- GitLab Shell
- GitLab Workhorse
- GitLab Elasticsearch Indexer

Gitaly Deprecations

Gitaly offers many customer facing features. As such, all deprecations to customer facing features will follow the standard GitLab feature deprecations guidance and announced within the deprecations documentation page.

Gitaly also offers many non-customer facing features, which are used by GitLab and other customers who directly interface with Gitaly. These Gitaly level deprecations will not be announced using the above methods as they are not designed for GitLab end users to interface with directly. Some examples of these non-customer facing features are storage level APIs, which should never be called by GitLab users.

Metrics

On gitlab.com

- Incidents (not all pages are incidents)
- Pages
- Global Apdex
- Alerts (S1/S2 are paging, S3/S4 are not)

Useful links

- Debugging Gitaly
- Actual pending Infradev issues (sort by group, focus on gitaly)
- Out of SLO Infradev issues
- Error budget
- MR review workload

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="gitaly" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="gitaly" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="gitaly" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="gitaly" >}}
{{< /tableau >}}
```

Team development

Onboarding

To complete team-specific onboarding, please file an issue [here](#).

Offboarding

Maintainer rights are revoked, and to remove the developer from the list of authorized approvers, remove them from the gl-gitaly GitLab.com group.

SECTION: engineering/infrastructure-platforms/data-access/gitaly/debug/_index.md

About this document

This document is intended for Gitaly engineers, to become familiar with GitLab's production layout and gain the ability to effectively debug production problems. While the focus is on SaaS, many of the skills transfer also to debugging self-managed instances.

Generic GitLab background

Skim / read the following, focusing on an overview then on Gitaly:

- Production Architecture
- Monitoring

Other useful links:

- Product sections, stages, groups, and categories
- Features by Group

Gitaly specific background

- Familiarize yourself with Gitaly's README
- Take a look at SRE's runbooks

Gitaly in Production

Both gitlab.com and Dedicated use Gitaly in "sharded" mode, that is, without Praefect (Gitaly Cluster).

Monitoring dashboards

We have some useful pre-built monitoring dashboards on GitLab's internal Grafana instance. All dashboards are listed in this folder. Please note that some of them are fairly outdated.

The following dashboards are most common:

- Gitaly: Overview. This dashboard contains cluster-wide aggregated metrics. It is used to determine the overall health of the cluster and make it easy to spot any outlier node.
- Gitaly: Host details. This dashboard contains more detailed metrics of a particular node.
- Gitaly Housekeeping statistics. This dashboard shows detailed operational information of Gitaly housekeeping feature.
- Gitaly: Rebalance dashboard: This dashboard shows the relative balance between Gitaly nodes. It is used to determine when we need to relocate the repositories of a node to others.

A Gitaly dashboard could be either auto-generated or manually drafted. We use Jsonnet (a superset of JSON) to achieve dashboards-as-code. The definitions of such dashboards are located in this folder. Recently, that's the recommended way to manage an observability dashboard. It allows us to use GitLab's built-in libraries, resulting in a highly standardized dashboard.

A standardized dashboard should have a top-level section containing environment filters, node filters, and useful annotations such as feature flag activities, deployments, etc. Some dashboards have an interlinked system that connects Grafana and Kibana with a single click.

Such dashboards usually include two parts. The second half contains panels of custom metrics collected from Gitaly. The first half is more complicated. It contains GitLab-wide indicators telling if Gitaly is "healthy" and node-level resource metrics. The aggregation and calculation are sophisticated. In summary, those dashboards tell us if Gitaly performs well according to predefined thresholds, . We could contact Observability team for any questions.

Some examples of using built-in dashboards to investigate production issues, from an Engineer's point of view:

- <https://gitlab.com/gitlab-com/gl-infra/production/-/issues/18156note1965772736>
- <https://gitlab.com/gitlab-com/gl-infra/production/-/issues/15980note1457815084>
- <https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/23532note1374642198>

Gitaly's Prometheus metrics

A panel in a dashboard is a visualization of the aggregated version of underlying metrics. We use Prometheus to collect metrics. To simplify, the Gitaly server exposes an HTTP server (code) that allows Prometheus instances to fetch metrics periodically.

In a dashboard, you can click on the top-right hamburger button and choose "Explore" to get access to the underlying metrics. Or you could use the Explore page to play with metrics.

Unfortunately, we don't have a curated list of all Gitaly metrics as well as their definition. So, you might need to look up their definition at multiple places. Here is the list of all Gitaly-related metrics. There are some sources

- Node-level or environmental metrics. Those metrics are powered by other systems that host the Gitaly process. They are not exposed by Gitaly but are very useful, for example: CPU metrics, memory metrics, or cgroup metrics.
- Gitaly-specific metrics. Those metrics are accounted for directly in the code. Typically, they have gitaly prefixes.
- Aggregated metrics, such as combining different metrics or downsizing metrics due to high cardinality issues. The list of Gitaly's aggregated metrics is listed in this file.

In the code, you'll see something like the following. Any registered metrics are available when Prometheus scrapes from the endpoint. Tracing those instances, you could find the usage of Gitaly-specific metrics.

```
go
repoCounter := counter.NewRepositoryCounter(cfg.Storages)
prometheus.MustRegister(repoCounter)

packObjectsServedBytes = promauto.NewCounter(prometheus.CounterOpts{
    Name: "gitalypackobjectsservedbytestotal",
    Help: "Number of bytes of git-pack-objects data served to clients",
})
```

A metric has a set of labels. GitLab adds the following set of labels to all metrics:

- env or environment: the environment, including but not limited to gprd, gstg, ops, to name a few.
- fqdn: the fully qualified domain name. As Gitaly runs on VMs now, this label is equivalent to the identity of the hosting node.
- region and zone: the region and zone of the node.
- stage: the current stage of the process, either main or cny.
- service/type: for Gitaly, it's always gitaly.

In the future, when Gitaly runs on K8s, we properly have more K8s-specific labels.

The query uses PromQL language. Some examples:

- Calculate the rate (ops/s) of pack-refs housekeeping task by node.
- Calculate the dropped pack-objects/RPC requests due to limited in the last 2 days
- Calculate inflight commands of gitaly-cny node. As you can see, there was as peak on

2024-06-17. It was when this incident occurs.

Debugging and performance testing tools

- gprcurl: curl like tool but for gPRC
- grpcui: lightweight postman like tool for gPRC
- hyperfine: a performance tool that can benchmarks over time
 - hyperfine can be used together with gprcurl to check the response time of a gPRC call

Enable Git trace

The gitalyloggittraces feature flag allows you to enable logging of Git trace2 spans for a specific user, group, or project. When enabled, Git outputs all trace2 spans to gRPC logs issued by that particular actor.

bash

```
/chatops run feature set --project=gitlab-org/gitlab gitalyloggittraces true
```

```
/chatops run feature set --user=myusername gitalyloggittraces true
```

Because of the frequency and noisiness of logs streaming in, customers should use a filter on server side logs to match the key and value of either:

- "msg":"Git Trace2 API"
- "component":"trace2hooks.logexporter"

For more information, see:

- Profile Git operations.
- Use ChatOps to enable and disable feature flags.

On the client side, for Git trace v1, the customer can enable GITTRACE variables, including:

bash

```
GITTRACE=true
```

```
GITTRACEPACKACCESS=true
```

```
GITTRACEPACKET=true
```

```
GITTRACEPERFORMANCE=true
```

For Git trace v2:

bash

Debugging git operations

If GITTRACE2PERFBRIEF or trace2.perfBrief is true, the time, file, and line fields are omitted.

```
GITTRACE2PERFBRIEF=1 GITTRACE2PERF=true git clone https://gitlab.com/gitlab-org/gitaly
```

```
GITTRACE2PERFBRIEF=1 GITTRACE2PERF=$(pwd)/git-perf.log git clone https://gitlab.com/gitlab-org/gitaly
```

Output git events in json format

```
GITTRACE2BRIEF=true GITTRACE2EVENT=$(pwd)/trace2.json git clone https://gitlab.com/gitlab-org/gitaly
```

Outputs can be configured in different formats:

bash

Normal format

```
export GITTRACE2=~/.log.normal
```

OR Performance format

```
export GITTRACE2PERF=~/.log.perf
```

OR Event format

```
export GITTRACE2EVENT=~/.log.event
```

OR JSON format

```
export GITTRACE2EVENT=~/.log.json
```

For more information, see:

- Git Trace v1 API

- Git Trace v2 API

strace

strace(1) a gitaly process:

shell

```
strace -fttTyyy -s 1024 -o /path/filename -p $(pgrep -fd, gitaly)
```

Or wrap a process to make it easy to strace, especially if it then spawns more processes:

shell

```
#!/bin/bash/sh
```

```
echo $(date)" $PPID $@" >> /tmp/gitlab-shell.txt
```

```
exec /opt/gitlab/embedded/service/gitlab-shell/bin/gitlab-shell-orig "$@"
```

```
strace -fttTyyy -s 1024 -o /tmp/sshdtrace-$PPID /opt/gitlab/embedded/service/gitlab-shell/bin/gitlab-shell-orig
```

strace parser is useful to make the results more readable.

fast-stats

fast-stats is a useful tool developed by Support to quickly pull statistics from GitLab logs.

Examples

To find in one interval of 60m duration what the top methods called are from the gitaly logs.

shell

```
fast-stats --interval 60m --limit 1 var/log/gitlab/gitaly/current
```

To find the top 10 User, Project, Client by Duration calling that method:

shell

```
grep PostUploadPackWithSidechannel var/log/gitlab/gitaly/current | ~/bin/fast-stats --interval 60m top
```

git

Snoop on Gitaly Git commands.

1. Stop Gitaly

1. Rename gitaly-git process `find /opt/gitlab/embedded/bin/gitaly-git-v -exec mv {} {}orig \;`

1. Create wrapper script for each git version. Make sure you replace `opt/gitlab/embedded/bin/gitaly-git-vX.XXorig` on the scrip with the right version.

shell

```
!/bin/bash
```

```
GIT="/opt/gitlab/embedded/bin/gitaly-git-vX.XXorig"
```

```
FILE="/tmp/gitaly-$(date +%Y-%m-%d@%H:%M)"
```

```
echo -e "\n$(date) $PPID $@" >> $FILE
```

```
exec $GIT "$@" | tee -a $FILE
```

```
echo -e "\n-----\n" >> $FILE
```

1. Make scripts executable `find /opt/gitlab/embedded/bin/gitaly-git-v -exec chmod 777 {} \;`

1. Start Gitaly

Log analysis

Kibana (Elastic) Dashboards

- gstg

- gprd

CPU and memory profiling

pprof metrics are exported to the GCP Cloud Profiler.

- gstg

- gprd

Note that Gitaly nodes are distributed across a number of GCP projects. You can use the project dropdown on the top nav bar to switch between the various `gitlab-gitaly-gstg-` and `gitlab-gitaly-gprd-` projects.

Capacity management

Gitaly team is responsible for maintaining reasonable serving capacity for gitlab.com.

We get alerts from Tamland if capacity runs low, see this issue comment.

Capacity planning documentation explains how this works in general.

SECTION: engineering/infrastructure-platforms/developer-experience/_index.md

Mission

Our mission is to empower developers to focus on innovation, build, and deliver high-quality products to our customers. We aim to achieve this through:

1. State-of-the-art developer tooling.
2. Robust and reliable test infrastructure.
3. Data-driven analysis for informed decision-making.
4. Streamlined release and test governance to ensure consistency, compliance, and quality across the development lifecycle.
5. Continuous performance profiling.

Team Structure

Infrastructure Platforms Department structure is documented in our handbook.

|

Developer Experience group structure

mermaid

graph TD

DE[Developer Experience group]

click DE "/handbook/engineering/infrastructure-platforms/developer-experience"

DE --> DA[Development Analytics]

click DA "/handbook/engineering/infrastructure-platforms/developer-experience/development-analytics"

DE --> DT[Developer Tooling]

click DT "/handbook/engineering/infrastructure-platforms/developer-experience/developer-tooling"

DE --> FR[Feature Readiness]

click FR "/handbook/engineering/infrastructure-platforms/developer-experience/feature-readiness"

DE --> PE[Performance Enablement]

click PE "/handbook/engineering/infrastructure-platforms/developer-experience/performance-enablement"

DE --> TG[Test Governance]

click TG "/handbook/engineering/infrastructure-platforms/developer-experience/test-

governance"

Team Members

Management team

{{< team-by-manager-role role="Director, Test Platform" >}}

Individual contributors

The following people are members of the Development Analytics group:

{{< team-by-manager-slug "ghosh-abhinaba" >}}

The following people are members of the Developer Tooling group:

{{< team-by-manager-slug "mgamea" >}}

The following people are members of the Feature Readiness group:

{{< team-by-manager-slug "ksvoboda" >}}

The following people are members of the Performance Enablement group:

{{< team-by-manager-slug "ksvoboda" >}}

The following people are members of the Test Governance group:

{{< team-by-manager-slug "amyphillips" >}}

Project Management

We use epics, issues, and issue/epic boards to organize our work, as they complement each other.

The structure of epics, and the responsibilities of the DRI are documented on the [The Infrastructure Platforms Project Management handbook](#) page

Weights

When estimating work, we use Fibonacci Series for weights and limit the highest number to 8. The definitions are as below:

Weight	Description
--------	-------------

--	--

----- -----	-----
-------------	-------

----- -----	-----
-------------	-------

-------	--

1 - Trivial	Simple and quick changes such as typo fixes, test tag updates, or trivial
-------------	---

documentation additions

|

| 2 - Small | Straight forward changes with no underlying dependencies needed, such as new tests that use existing factories or page objects

|

| 3 - Medium | Well understood changes with a few dependencies. Few surprises can be expected, such as new tests that require new factories or page objects/components

|

| 5 - Large | A task that will require some investigation and research, in addition to the above weights, such as tests that need framework level changes which can impact other parts of the test suite |

| 8 - X-large | A very large task that will require much investigation and research. Pushing initiative level

|

| 13 or more | Please break the work down further, we do not use weights higher than 8. |

SECTION: engineering/infrastructure-platforms/developer-experience/bug-prioritization.md

Overview

This page describes the bug prioritization process performed by the quality engineering sub-department as part of the cross-functional prioritization process.

An overview of the full process can be found on the cross-functional prioritization handbook page.

Timeline

As defined in our Product Development Timeline, there are specific deadlines for prioritization. Given a milestone m that will be shipped in month M (on the release date):

- By the Monday, 19 days before the milestone begins:
 - Bug Prioritization triage report is created with a list of prioritized type::bug issues to the product managers of each groups.
- By the Monday, 12 days before the milestone begins:
 - the quad for each counterpart group (PMs, EMs, UX, and Quality) discusses which issues to include in the upcoming milestone.
 - the product manager, taking into consideration prioritization input from the quad, creates a plan of issues for the upcoming milestone.

Bug Prioritization triage report

An automated Bug Prioritization triage report is created for each group on a monthly basis containing the Top 10 open issues of ~"type::bug". The report is generated with issues that are prioritized based on the below conditions:

1. Severity based on label severity::
1. Security vulnerabilities based on label bug::vulnerability

1. Customer issues based on label customer
1. Age of the issues(oldest)

Bug Prioritization triage report are automatically assigned to specific PMs, EMs of each group, listed in the group definition file, or directly in the triage report policy files policy YAML files.

To change who an issue gets assigned to, open a merge request for the above files. If the group definition file is changed, we'll need to run some scripts to update the generated files as well.

Bug Prioritization Dashboard

The bug prioritization dashboard can also be used to put together a suggested list of bugs for each group.

This dashboard should be revisited for each milestone, as the data will change over time.

As a general guideline, the total amount of bugs proposed should match the burndown plan from the dashboard for your specific group.

This total can be found under Total Bugs in the chart titled Milestone planning (bug counts to schedule).

It is also helpful to include the Total Bug % as well, which can be found in the Milestone planning (bug % of scheduled issues) chart.

This metric can help groups aim toward scheduling the appropriate percentage of bugs if they do not have the capacity to work on every bug listed.

This process is not intended to be one-size-fits-all, and groups are encouraged to find what best works for the team.

Dashboard Walkthrough

This table includes a list of the charts that can be found on the bug prioritization dashboard and a description of what each one represents.

The bug prioritization dashboard walkthrough video also offers an overview of the dashboard with examples.

Chart Name	Description
------------	-------------